

### 1. Cache Miss Handling & Write Policies

Read Miss:

When a read miss occurs, the cache controller checks the dirty bit of the existing block in the indexed cache line. If the block is dirty, a write-back is performed first, writing the modified block to the main memory. Then, the requested missing block is fetched from the main memory and placed into the cache, updating the valid bit to 1 and dirty bit to 0. Finally, the requested word is sent to the CPU.

Write Miss:

Similar to a read miss, if the existing block is dirty, it is written back to the main memory. The new missing block is then fetched from the main memory and allocated in the cache. Once the block is in the cache, the CPU's write access proceeds as a 'write-hit', modifying the specific word in the cache and setting the dirty bit to 1.

Write Policies:

- Write-Back: The cache is updated on a write, and the dirty bit is set. Main memory is only updated when the dirty block is evicted. This system implements a write-back policy.
- Write-Through: Every write updates both the cache and the main memory simultaneously, ensuring memory is always up-to-date but incurring high memory traffic.

Flowchart Process:

1. CPU requests access -> 2. Check Tag & Valid bit.
3. If Hit -> Serve request (read sends data, write modifies cache & sets dirty).
4. If Miss -> Check Dirty bit of existing block.
5. If Dirty == 1 -> Write-back existing block to memory.
6. Fetch new block from memory -> Update cache (Valid=1, Dirty=0).
7. Serve original request.

### 2. Cache Mapping Mechanism & Address Format

The required cache mapping mechanism is Direct-Mapped Cache. Since the cache has 8 blocks and each block is 4 Bytes, the 8-bit memory address is partitioned as follows:

- Offset: 2 bits (Bits 1:0) to select 1 of 4 bytes within a block.
- Index: 3 bits (Bits 4:2) to select 1 of 8 blocks in the cache.
- Tag: 3 bits (Bits 7:5) to identify the memory origin of the block.

Bit Format:

[ 7 | 6 | 5 ] [ 4 | 3 | 2 ] [ 1 | 0 ]

## CO2070 Lab 6 Answers

Tag      Index      Offset

### 3. Cache Table Trace

Final Cache State after the given instructions:

Index 0:

- Valid: 1
- Dirty: 1
- Tag: 001 (binary for 1)
- Data: [Byte0=0x08, Byte1=0x00, Byte2=0x00, Byte3=0x00] (from memory address 0x20)

Index 1:

- Valid: 1
- Dirty: 1
- Tag: 000 (binary for 0)
- Data: [Byte0=0x00, Byte1=0x00, Byte2=0x00, Byte3=0x08] (from memory address 0x07)

Indices 2 through 7:

- Valid: 0
- Dirty: 0
- Tag: 000
- Data: [0x00, 0x00, 0x00, 0x00]

### 4. FSM Skeleton Completion

To complete the FSM flowchart image for the Cache Controller:

1. Add a new state bubble named 'MEM\_WRITE'.
2. Draw an arrow from 'IDLE' to 'MEM\_WRITE' with the condition: '(read=1 || write=1) && dirty=1 && hit=0'.
3. Draw a self-looping arrow on 'MEM\_WRITE' with the condition: 'mem\_busy = 1'.
4. Draw an arrow from 'MEM\_WRITE' to 'MEM\_READ' with the condition: 'mem\_busy = 0'.
5. Ensure the arrow from 'MEM\_READ' back to 'IDLE' has the condition: 'mem\_busy = 0'.
6. Update the self-loop on 'IDLE' to also include the hit condition: '(read=0 && write=0) || hit=1'.